

ADITYA COLLEGE OF ENGINEERING AND TECHNOLOGY

Surampalem, Andhra Pradesh

Department of Information Technology

INTERNSHIP / ACADEMIC PROJECT REPORT

EduGenie

Google Gemini Powered Learning Assistant

A Full-Stack AI-Powered Educational Platform Built with FastAPI and Google Gemini AI

Submitted by

DEVARA RAVITEJA (Roll No. 23P31A1239)

DUNABOYINA JYOSHNA (Roll N0. 23P31A1240)

College	Aditya College of Engineering and Technology
Department	Information Technology
Academic Year	2026

Table of Contents

1. Project Overview	3
2. System Architecture	4
3. Technology Stack.....	5
4. Project Workflow	6
5. Module Descriptions	7
6. API Architecture	13
7. Backend Workflow	14
8. User Interface Design	15
9. Local Setup	16
10. Testing.....	17
11. Challenges Faced	18
12. Future Enhancements.....	19
13. Conclusion	20
14. References.....	21

1. Project Overview

Problem Statement

Most existing e-learning platforms deliver static, one-size-fits-all educational content such as pre-recorded videos, fixed reading material, and generic assessments. They do not adapt to an individual learner's pace, prior knowledge, or specific doubts, and they offer little to no real-time, conversational assistance. As a result, students who need instant clarification of a concept, a personalized revision plan, or a quick way to test their understanding are forced to depend on external tutors, scattered internet searches, or trial-and-error learning, which is inefficient and often discouraging.

Proposed Solution

EduGenie addresses this gap by integrating Google's Gemini large language model with a lightweight, high-performance FastAPI backend to deliver an interactive, all-in-one educational assistant. The platform allows a learner to converse naturally with an AI tutor, request simplified explanations of difficult concepts, generate topic-wise multiple-choice quizzes, summarize lengthy study material, and receive a structured, time-bound learning roadmap for any subject — all from a single, responsive web interface.

Objectives

- To design and develop a full-stack AI-powered learning assistant accessible through a web browser.
- To integrate the Google Gemini API for natural-language understanding and generation across multiple learning use-cases.
- To build a clean, modular FastAPI backend that exposes well-defined REST endpoints for each learning feature.
- To provide a distraction-free, responsive, and accessible front-end experience using HTML5, CSS3, and vanilla JavaScript.
- To reduce the time and effort a student spends searching for explanations, revision material, and structured study plans.

Benefits

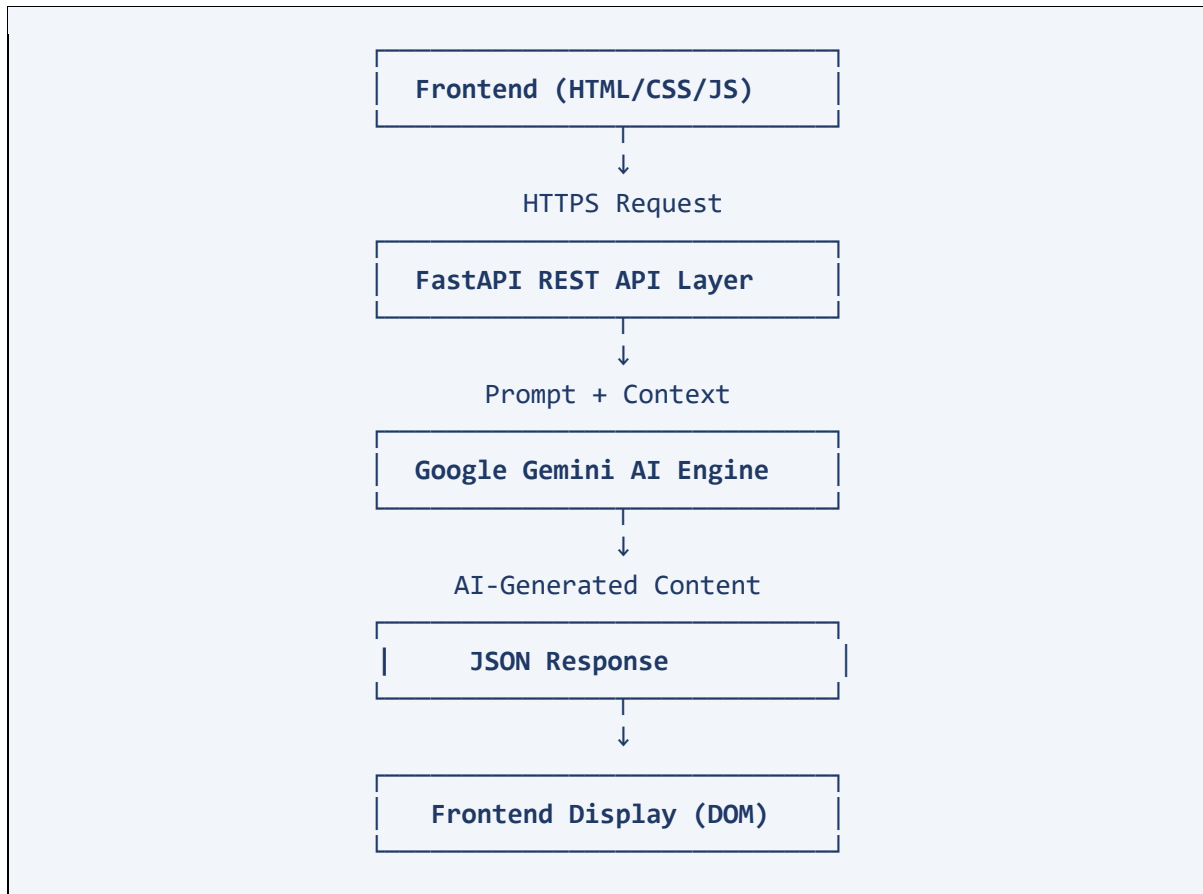
- Instant, on-demand academic support without dependency on a live tutor.
- Personalized explanations that adapt to the learner's selected difficulty level.
- Self-assessment through automatically generated quizzes with instant feedback.
- Significant reduction in reading and revision time through AI summarization.
- A clear, structured roadmap that removes the guesswork from self-paced learning.

Expected Outcomes

Upon completion, EduGenie is expected to function as a dependable digital study companion capable of handling everyday academic queries, generating exam-style practice questions, condensing long-form content into digestible notes, and producing week-wise learning plans — thereby improving learning efficiency, engagement, and outcomes for students across subjects and skill levels.

2. System Architecture

EduGenie follows a clean, layered client-server architecture. The browser-based frontend communicates with a FastAPI backend over REST APIs; the backend, in turn, communicates with the Google Gemini API to generate AI-driven content, and returns structured JSON responses back to the frontend for rendering.



This layered design keeps concerns cleanly separated: the frontend focuses purely on presentation and user interaction, the FastAPI layer handles routing, validation, and business logic, and the Gemini AI engine performs the actual natural-language reasoning. Configuration values such as API keys and model parameters are isolated using Pydantic Settings and environment variables, keeping the system secure and easy to maintain.

3. Technology Stack

The table below summarizes the technologies used across every layer of the EduGenie application.

Layer	Technology	Purpose
Frontend	HTML5, CSS3, JavaScript (Vanilla)	Structure, styling, and interactivity of the user interface
Backend	Python, FastAPI, Uvicorn	REST API development and high-performance ASGI serving
Artificial Intelligence	Google Gemini API	Natural language understanding, generation, and reasoning
Styling	Custom CSS3 (Flexbox / Grid)	Responsive, modern, card-based UI layout
API Communication	Fetch API (JavaScript)	Asynchronous communication between frontend and backend
Configuration	python-dotenv, Pydantic Settings	Secure environment-variable and settings management
Development Tools	VS Code, Chrome DevTools	Coding, debugging, and UI inspection
Version Control	Git, GitHub	Source code management and collaboration
Deployment	Localhost / Cloud (Uvicorn server)	Running and hosting the application

4. Project Workflow

The overall workflow of EduGenie, from the moment a user opens the application to the moment they receive AI-generated results, follows a consistent and predictable pattern across every module:

- The user opens the EduGenie web application in a browser, landing on the Home page.
- The user navigates to the desired module using the top navigation bar — AI Chat, Explainer, Quiz Gen, Summarizer, or Roadmap.
- The user fills in the required input fields for that module (for example, a topic, a difficulty level, or source text).
- On clicking the action button, the frontend JavaScript validates the input and sends an asynchronous request to the corresponding FastAPI endpoint.
- The FastAPI backend constructs a structured prompt and forwards it to the Google Gemini API.
- Gemini processes the request and returns AI-generated content, which the backend formats into a clean JSON response.
- The frontend receives the JSON response and dynamically renders it in the results panel without a full page reload.
- The user can copy, review, or act upon the generated content, or submit a new request to continue learning.

5. Module Descriptions

This section describes each functional module of EduGenie in detail, including its purpose, internal workflow, key features, advantages, and a representative screenshot.

5.1 Home Page

The Home page serves as the entry point of EduGenie. It introduces the platform to first-time visitors, communicates its value proposition, and provides quick navigation to every learning module through a modern, responsive interface.

Workflow

- 1. User lands on the Home page after opening the application.
- 2. A welcome banner and feature highlights summarize what EduGenie offers.
- 3. The navigation bar allows the user to jump directly to any module.

Key Features

- Clean, modern hero section with a clear call-to-action.
- Feature cards summarizing Smart Explanations and Adaptive Quizzes.
- Fully responsive navigation bar with active-state highlighting.
- Consistent branding and visual identity across the platform.

Advantages

- Creates a strong first impression and improves user retention.
- Reduces the learning curve by clearly signposting available features.
- Provides a consistent visual anchor for every other module.

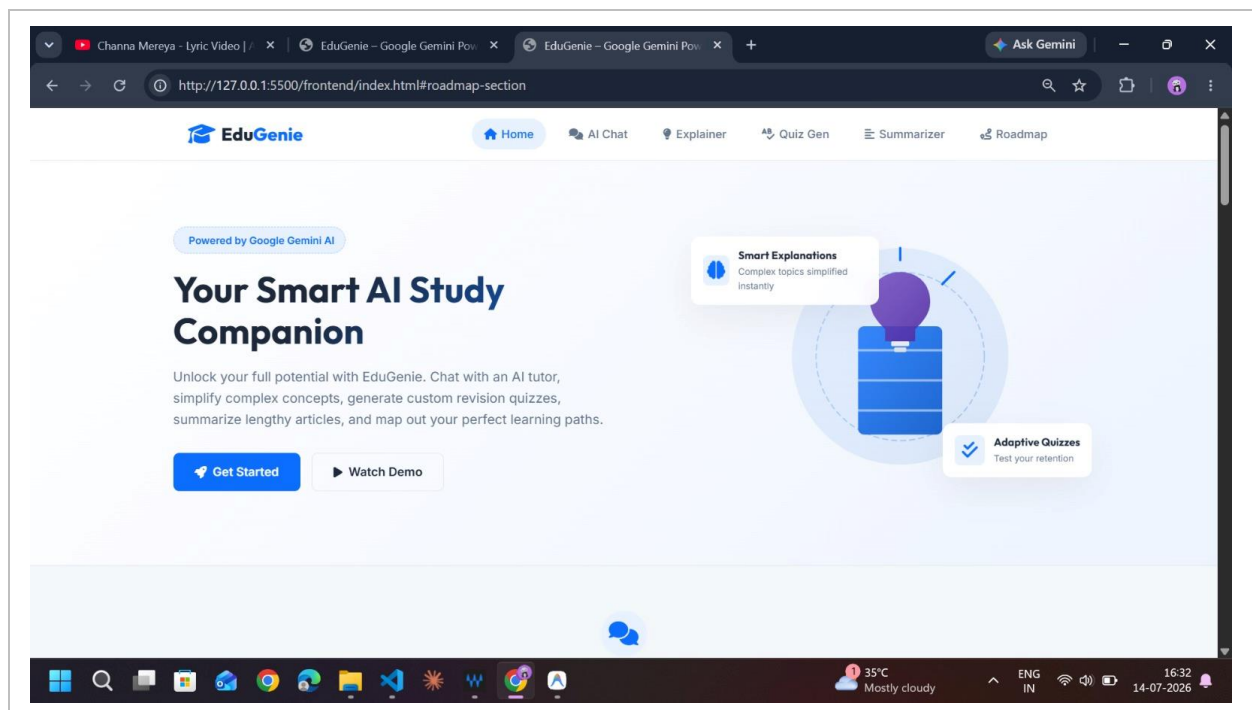


Figure 1 – EduGenie Home Page

5.2 AI Chat Assistant

The AI Chat Assistant allows students to converse naturally with an AI tutor. It is designed to answer academic questions, clarify doubts, and hold a context-aware conversation across multiple turns.

Workflow

- 1. User types a question or doubt into the chat input box.
- 2. The frontend sends the message, along with recent conversation history, to the /chat endpoint.
- 3. FastAPI forwards the conversation context to Gemini, which generates a relevant, student-friendly reply.
- 4. The response is appended to the chat window in real time.

Key Features

- Natural, human-like conversational flow.
- Context retention across multiple chat turns.
- Academic question answering across subjects.
- Fast, asynchronous response rendering.

Advantages

- Feels like chatting with a personal tutor available 24/7.
- Removes the intimidation factor of asking basic doubts.
- Encourages active engagement rather than passive reading.

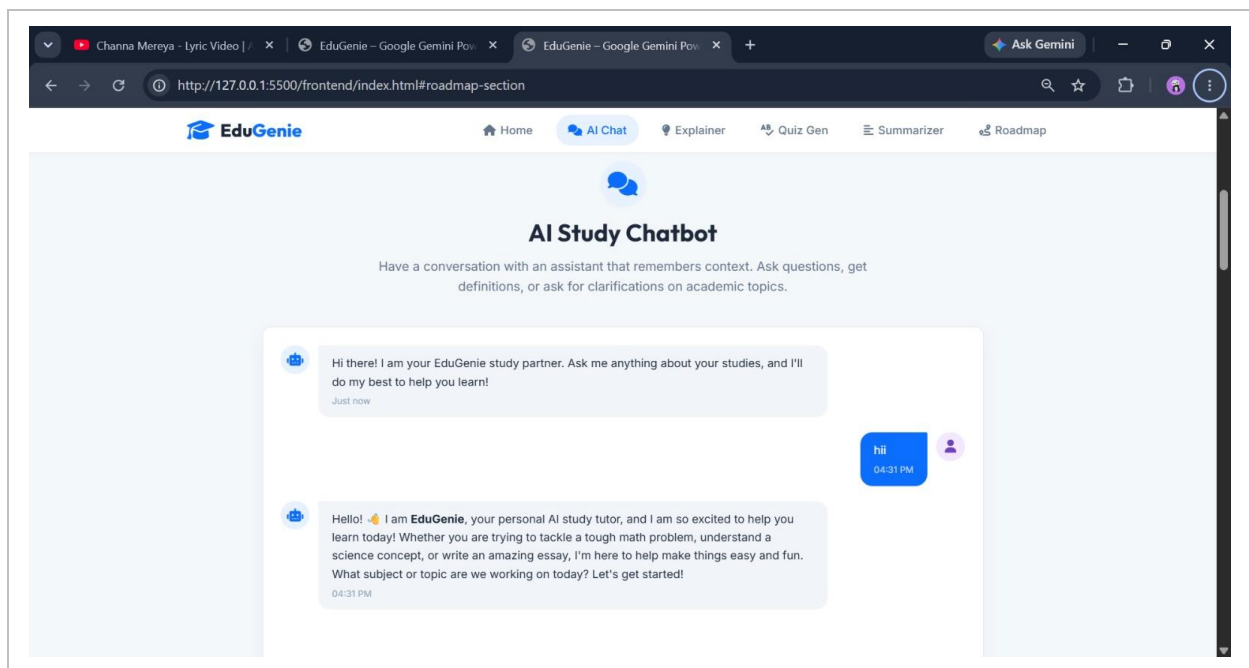


Figure 2 – AI Chat Assistant

5.3 Concept Explainer

The Concept Explainer simplifies difficult academic topics based on the learner's chosen background level, ranging from beginner to advanced, making dense subject matter easy to digest.

Workflow

- 1. User enters a topic (for example, 'Python') and selects a target audience level.
- 2. On clicking 'Simplify Concept', the frontend calls the /explain endpoint with the topic and level.

- 3. Gemini generates a plain-language explanation along with a relatable, everyday analogy.
- 4. The explanation, analogy, and key points are rendered in a structured results card.

Key Features

- Adjustable explanation depth based on learner level.
- Real-life analogies that make abstract ideas concrete.
- Clearly separated key points for quick revision.
- Copy-to-clipboard support for saving notes.

Advantages

- Bridges the gap between textbook language and everyday understanding.
- Speeds up comprehension of unfamiliar or intimidating topics.
- Supports learners of varying academic backgrounds with one tool.

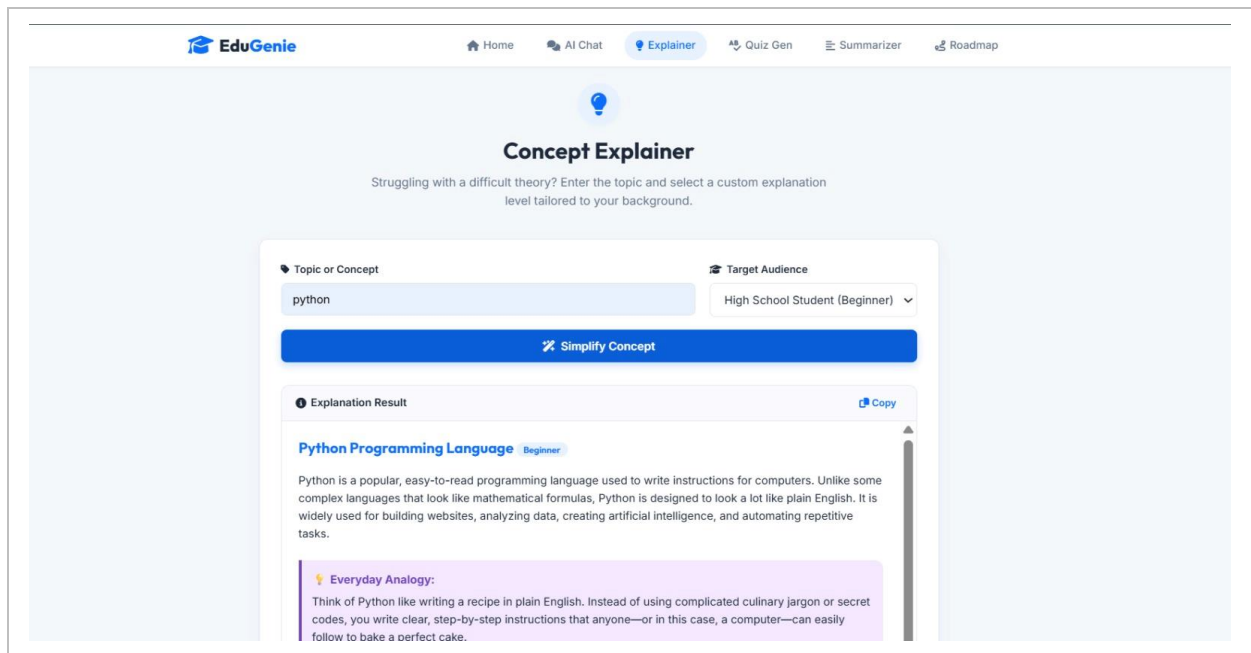


Figure 3 – Concept Explorer

5.4 Quiz Generator

The Quiz Generator produces topic-based multiple-choice quizzes on demand, allowing learners to test their understanding immediately after studying a concept.

Workflow

1. User enters a quiz topic and selects the desired number of questions.
2. The frontend sends this input to the /quiz endpoint.
3. Gemini generates a set of questions, each with four options, a correct answer, and supporting educational content.
4. The quiz is rendered interactively, allowing the user to select an answer and see instant, color-coded feedback.

Key Features

- Dynamically generated, topic-specific multiple-choice questions.
- Four-option format matching standard academic assessments.
- Instant visual feedback on correct and incorrect answers.

- Adjustable question count to control quiz length.

Advantages

- Enables quick self-assessment without manual question creation.
- Reinforces learning through active recall rather than passive review.
- Highly reusable across any subject or topic the learner is studying.

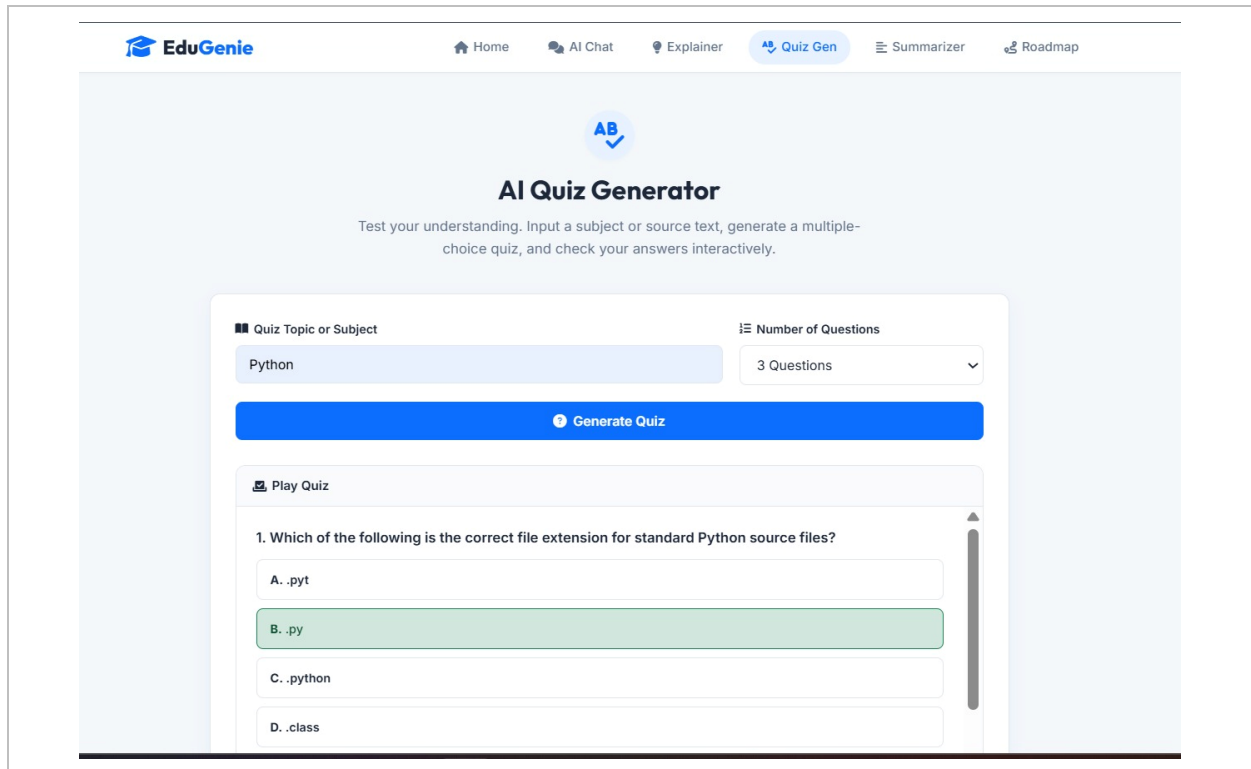


Figure 4 – Quiz Generator

5.5 Smart Summarizer

The Smart Summarizer condenses long-form educational content — such as articles, textbook excerpts, or research material — into short, easy-to-read summaries.

Workflow

1. User pastes source text into the input area and selects a summary style, such as 'Short & Bulleted'.
2. On clicking 'Summarize', the frontend sends the text to the /summarize endpoint.
3. Gemini analyzes the content and produces a concise summary with key bullet points.
4. The result is displayed in a dedicated summary panel with a one-click copy option.

Key Features

- Handles long passages of unstructured text.
- Produces both a short overview and detailed key points.
- Multiple summary styles to suit different reading needs.
- One-click copy for quick reuse in notes.

Advantages

- Drastically reduces reading and revision time.
- Helps learners extract the most important information quickly.

- Useful for exam preparation and last-minute revision.

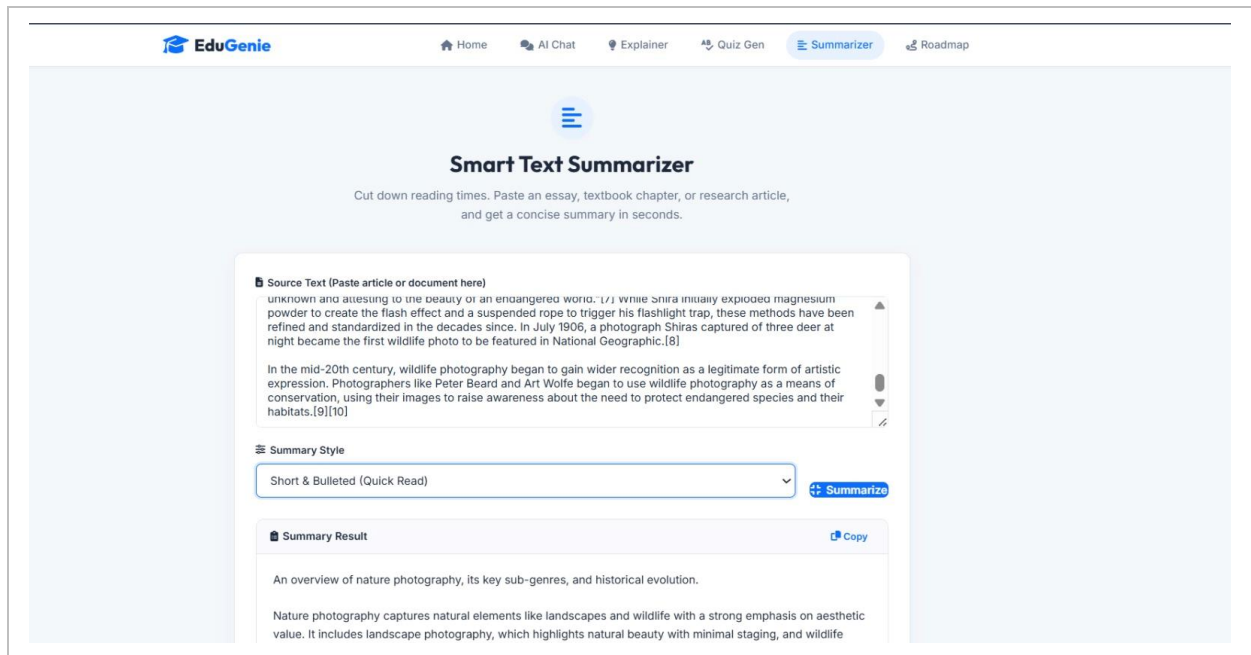


Figure 5 – Smart Summarizer

5.6 Learning Roadmap Generator

The Roadmap Generator builds a personalized, week-wise learning plan for any subject or career goal, complete with milestones, curated resources, and a clear progress path.

Workflow

1. User enters a subject or career goal (for example, 'DSA') and selects the allocated duration.
2. On clicking 'Generate Roadmap', the frontend calls the /roadmap endpoint.
3. Gemini creates a structured, timeline-based curriculum covering topics, milestones, and resources.
4. The roadmap is displayed as an interactive, scrollable timeline with curated resource links for each stage.

Key Features

- Duration-based roadmap customization (for example, crash course vs. extended plan).
- Clear weekly milestones with topic breakdowns.
- Curated external resource links for deeper study.
- Visually organized timeline interface.

Advantages

- Removes the guesswork from self-paced learning.
- Keeps learners accountable with a clear, time-bound structure.
- Adapts to both short crash-course goals and long-term mastery plans.

The screenshot shows the EduGenie website's 'Personalized Learning Roadmap' feature. At the top, the EduGenie logo is on the left, and navigation links for Home, AI Chat, Explorer, Quiz Gen, Summarizer, and Roadmap are on the right. The 'Roadmap' link is highlighted. Below the navigation bar, a blue icon of two people is centered above the title 'Personalized Learning Roadmap'. A subtitle reads: 'Create a structured step-by-step curriculum to learn any subject, complete with milestones, duration suggestions, and resource lists.'

The main form has two input fields at the top: 'Subject or Career Goal' with the value 'DSA' and 'Allocated Duration' with a dropdown menu showing '4 Weeks (Crash Course)'. Below these is a large blue button labeled 'Generate Roadmap'. Underneath the button is a section titled 'Generated Roadmap Curriculum'. It features a vertical timeline on the left with a blue circle at the top and bottom. The first item on the timeline is 'Linear Data Structures & Complexity Analysis', labeled 'Week 1'. The description for this item is: 'Learn the basics of Big O notation and master linear data structures including Arrays, Linked Lists, Stacks, and Queues. Understand their memory allocation and time complexities for basic operations.' Below the description, under the heading 'Curated Study Resources:', there are two links: 'Introduction to Algorithms by CLRS' and 'GeeksforGeeks - Data Structures Guide'.

Figure 6 – Learning Roadmap Generator

6. API Architecture

EduGenie's backend exposes a small set of clearly defined REST endpoints, each corresponding to one learning module. Every endpoint accepts a JSON request body and returns a structured JSON response generated with the help of the Google Gemini API.

Endpoint	Method	Purpose
/chat	POST	Handles conversational academic queries and returns context-aware AI responses
/explain	POST	Generates a simplified explanation and analogy for a given topic and audience level
/quiz	POST	Generates a multiple-choice quiz for a given topic and question count
/summarize	POST	Produces a condensed summary and key points from pasted source text
/roadmap	POST	Creates a personalized, duration-based learning roadmap for a subject

All endpoints follow a consistent request-response contract: the frontend sends a JSON payload containing user input, FastAPI validates it using Pydantic models, forwards a well-structured prompt to Gemini, and returns the AI-generated output as clean, parsed JSON ready for direct rendering on the frontend.

7. Backend Workflow

Every request made from the EduGenie frontend passes through a consistent backend pipeline before an AI-generated response is returned to the user.



The FastAPI router receives the incoming request and validates it against a Pydantic schema. The request is then passed to a dedicated Gemini service layer, which formats an appropriate prompt, calls the Google Gemini API, and parses the returned content. The parsed result is wrapped into a standard JSON response and sent back to the frontend, where it is rendered directly into the relevant results panel without a page reload.

8. User Interface Design

EduGenie's interface is designed to feel calm, modern, and distraction-free, keeping the learner's focus on content rather than clutter.

Typography

A clean, legible sans-serif typeface is used throughout the application, with a clear visual hierarchy between headings, subheadings, and body text to guide the user's attention naturally.

Colour Palette

The interface uses a calm blue-and-white palette — blue for primary actions, navigation highlights, and branding, and soft off-white and light-grey backgrounds for content areas — creating a professional, education-appropriate look and feel.

Cards

Each module presents its inputs and results within rounded, shadowed card components. This consistent card-based layout makes the interface predictable and easy to scan across all six modules.

Animations

Subtle hover states, smooth panel transitions, and lightweight loading indicators are used to provide feedback during asynchronous API calls without distracting the learner.

Navigation

A persistent top navigation bar with clearly labeled icons for Home, AI Chat, Explainer, Quiz Gen, Summarizer, and Roadmap allows the user to move between modules in a single click, with the active module visually highlighted.

Accessibility

The interface maintains sufficient colour contrast, legible font sizing, clearly labeled input fields, and a responsive layout that adapts cleanly across desktop and mobile screen sizes.

9. Local Setup

The following steps describe how to set up and run EduGenie on a local development machine.

Installation Steps

- Clone the repository: `git clone <repository-url>`
- Navigate into the project directory: `cd EduGenie`
- Create a virtual environment: `python -m venv venv`
- Activate the virtual environment (`venv\Scripts\activate` on Windows, `source venv/bin/activate` on macOS/Linux)
- Install backend dependencies: `pip install -r requirements.txt`
- Create a `.env` file in the project root and add your Google Gemini API key.
- Run the FastAPI backend server: `uvicorn main:app --reload`
- Open the `frontend/index.html` file in a browser, or serve it using a local development server.

Project Structure

```

EduGenie/
├── backend/
│   ├── main.py
│   ├── config.py
│   ├── routers/
│   │   ├── chat.py
│   │   ├── explain.py
│   │   ├── quiz.py
│   │   ├── summarize.py
│   │   └── roadmap.py
│   └── services/
│       ├── gemini_service.py
│       └── .env
├── frontend/
│   ├── index.html
│   ├── css/
│   │   └── style.css
│   ├── js/
│   │   └── app.js
├── requirements.txt
└── README.md

```

10. Testing

EduGenie was tested across multiple layers to ensure functional correctness, reliable API behavior, and a smooth user experience.

Functional Testing

Each module — Chat, Explainer, Quiz Generator, Summarizer, and Roadmap Generator — was tested individually with a variety of topics and inputs to confirm that it produced relevant, well-structured AI responses.

API Testing

Every FastAPI endpoint was tested for correct request validation, appropriate error handling for missing or invalid fields, and consistent JSON response formatting.

UI Testing

The interface was tested across different screen sizes and browsers to confirm responsive behavior, correct rendering of dynamic content, and smooth loading states during API calls.

Integration Testing

End-to-end flows — from user input on the frontend, through the FastAPI backend, to the Gemini API and back — were verified to ensure data passed correctly through every layer without loss or corruption.

Expected Results

- All modules return accurate, contextually relevant AI-generated content.
- API endpoints respond within an acceptable time frame under normal load.
- The interface remains responsive and error-free across supported browsers and devices.
- Invalid or empty inputs are handled gracefully with clear user feedback.

11. Challenges Faced

- Integrating the Google Gemini API required careful prompt design to ensure consistent, well-structured responses for each module.
- Setting up clean backend routing in FastAPI for multiple distinct AI-powered endpoints while keeping the codebase modular and maintainable.
- Correctly configuring FastAPI with Uvicorn and managing CORS to allow smooth communication with the frontend.
- Securely managing environment variables and API keys using python-dotenv and Pydantic Settings without exposing sensitive data.
- Handling Gemini API quota and rate-limit constraints during repeated testing and development.
- Troubleshooting initial deployment attempts, including environment configuration differences between local and hosted environments.

12. Future Enhancements

- User authentication and personalized learner accounts.
- Progress tracking dashboards across quizzes and roadmaps.
- Voice assistant integration for hands-free interaction.
- Speech-to-text input for the chat assistant.
- A dark mode theme for extended study sessions.
- PDF upload support for direct summarization of documents.
- Retrieval-Augmented Generation (RAG) for grounding responses in specific course material.
- Multi-language support for a wider range of learners.
- A dedicated student dashboard consolidating all learning activity.
- Full cloud deployment for public accessibility and scalability.

13. Conclusion

EduGenie successfully demonstrates how a modern AI model like Google Gemini can be combined with a lightweight FastAPI backend and a clean, responsive frontend to create a genuinely useful educational assistant. By bringing conversational support, concept explanation, quiz generation, summarization, and personalized roadmap planning into a single unified platform, EduGenie addresses a real gap in how students currently access and consume learning support.

The project reflects a complete full-stack development workflow — from designing REST APIs and integrating a third-party AI service, to building an accessible, responsive user interface — and lays a solid foundation for future enhancements such as authentication, progress tracking, and cloud deployment. Overall, EduGenie stands as a practical, extensible example of applying generative AI to solve a meaningful, everyday problem in education.

14. References

- FastAPI Official Documentation — <https://fastapi.tiangolo.com>
- Google Gemini API Documentation — <https://ai.google.dev>
- MDN Web Docs — HTML — <https://developer.mozilla.org/en-US/docs/Web/HTML>
- MDN Web Docs — CSS — <https://developer.mozilla.org/en-US/docs/Web/CSS>
- MDN Web Docs — JavaScript — <https://developer.mozilla.org/en-US/docs/Web/JavaScript>
- Python Official Documentation — <https://docs.python.org>
- GitHub — <https://github.com>
- Google AI Studio — <https://aistudio.google.com>